

CulturaX

Technical Architecture & Implementation Report
AR-Based Egyptian Heritage Tourism Application

Version 2.0 | June 2026
Flutter 3.11.1 • ARKit / ARCore • Google Cloud Vision • Firebase

Table of Contents

1 Project Overview	3
2 System Architecture	3
2.1 Technology Stack	3
2.2 Application Layers	4
3 Monument Detection Pipeline	4
4 Augmented Reality Experience	5
4.1 AR Session Lifecycle	5
4.2 3D Model System	6
4.3 Fun Facts Carousel	6
5 Monument Database	7
5.1 MonumentInfo Data Model	7
5.2 Registered Monuments	7
5.3 NEW: Pyramid of Khafre	8
6 Gamification System	9
7 Asset Structure	10
8 Key Screens & Navigation	10
9 Technical Implementation Details	11
10 Dependencies	12
11 Future Extension Guide	13

1. Project Overview

CulturaX is a Flutter-based cross-platform mobile application targeting iOS and Android that uses Augmented Reality and AI-powered monument recognition to create immersive educational experiences at Egyptian heritage sites. Users point their smartphone camera at any registered monument; the app identifies it via the Google Cloud Vision API and immediately launches a real-time AR session in which a procedurally-generated 3D model of that monument is placed on a physical surface using ARKit (iOS) or ARCore (Android). The experience layers historical facts, audio narration, and gamified treasure-hunt missions on top of the live camera feed.

Core Value Proposition
Scan any Egyptian landmark in seconds with AI landmark recognition.
Place a 3D monument model on your desk, floor, or tabletop via AR.
Swipe through 8 curated fun facts per monument with animated cards.
Earn points, badges, and artifacts through treasure hunt missions.
Listen to audio narration of each monument's history via TTS.

2. System Architecture

2.1 Technology Stack

Technology	Role / Notes
Flutter 3.11.1 / Dart 3.11.1	Cross-platform UI framework — single codebase for iOS, Android, Web
ar_flutter_plugin 0.7.3	AR wrapper — ARKit on iOS, ARCore on Android; plane detection & GLB nodes
Google Cloud Vision API	LANDMARK_DETECTION — identifies monuments from camera images via REST
Firebase Auth + Firestore	User authentication and persistent game progress storage
flutter_tts 4.2.0	Text-to-Speech audio narration for monument history
Provider 6.1.5	State management — GameProgressService, AuthService
confetti 0.8.0	Particle celebration effect on AR model placement
camera 0.11.1	Native camera controller for high-res JPEG capture

2.2 Application Layers

The application follows a four-layer architecture ensuring clean separation between presentation, business logic, data persistence, and native platform capabilities.

■ Presentation Layer

Flutter widgets, animated screens (SphinxARScreen, KhafrePyramidARScreen, ARMonumentScannerScreen, TreasureHuntScreen, ARExperienceScreen). Manages UI state via `setState()` and `AnimationController`.

■ Business Logic Layer

Service classes: `MonumentDataService` (registry + fuzzy matching), `GoogleVisionService` (Vision API), `GameProgressService` (points/badges), `AudioNarrationService` (TTS).

■ Data Layer

Firebase Firestore (user progress), `SharedPreferences` (local cache), local asset bundles (images, GLB models, labels).

■ Native Layer

ARKit (iOS) and ARCore (Android) via `ar_flutter_plugin` bridge. camera package for JPEG capture. `getUserMedia()` on web.

3. Monument Detection Pipeline

The detection pipeline transforms a camera image into a routed AR screen in six discrete steps. Each step has a defined fallback to ensure a graceful user experience even without connectivity.

Step 1 — Image Capture

User taps the gold scan button. On iOS/Android, `CameraController` (`ResolutionPreset.high`, `JPEG`) calls `takePicture()`. On Web, `WebCamController` wraps `getUserMedia()` and returns a `Uint8List` frame. Both converge at `_scan(Uint8List bytes)`, which also caches the bytes in `_lastCapturedBytes` for the AR hotspot overlay.

Step 2 — Vision API Request

`GoogleVisionService` encodes the image as base64 and POSTs to `vision.googleapis.com/v1/images:annotate` with feature `LANDMARK_DETECTION`, `maxResults=5`, and a 15-second timeout.

Step 3 — Response Parsing

The JSON response is parsed for landmark annotations. Each annotation with `score >= 0.55` is valid. The top result becomes a `DetectedLandmark` `{name, confidence, latitude?, longitude?}`.

Step 4 — 4-Pass Fuzzy Matching

`MonumentDataService.findByName()` runs four passes:

Pass 1 — Exact alias match (case-insensitive)

Pass 2 — Partial containment (query contains alias OR alias contains query)

Pass 3 — Significant word match (name words >4 chars in query)

Pass 4 — Reverse word match (query words >4 chars in any alias)

First match wins, preventing false positives.

Step 5 — AR Screen Routing

`_resolveARScreen(MonumentInfo)` maps `monument.id` to a `Widget`:

`khafre_pyramid` → `KhafrePyramidARScreen` [NEW]

`sphinx` → `SphinxARScreen`

(any other id) → `SphinxARScreen` fallback.

`Navigator.push()` launches the resolved screen.

Step 6 — API Unavailable Fallback

If the Vision API errors or returns nothing above threshold, `GoogleVisionService` returns a simulation: images > 65 KB → `DetectedLandmark('The Great Sphinx', 0.96)`, ensuring the demo always works offline.

4. Augmented Reality Experience

4.1 AR Session Lifecycle

1. Initialization	ARView mounts and calls onARViewCreated, receiving session/object/anchor managers. Initialized with showFeaturePoints=true, showPlanes=true.
2. Plane Detection	ARCore/ARKit scans for flat surfaces (horizontal + vertical). An animated pulsing ring prompts the user to scan slowly.
3. Hit Testing	User taps screen. onPlaneOrPointTap fires with List. First result of type plane is selected.
4. Anchor Creation	ARPlaneAnchor built from hit.worldTransform, added via addAnchor(). On failure an error hint is shown.
5. Node Placement	ARNode(type: localGLTF2, uri: '{model}.glb', scale: Vector3) added via addNode(node, planeAnchor: anchor).
6. Celebration	HapticFeedback.heavyImpact(), confetti.play(), welcome banner ScaleTransition for 3-4s, then auto-dismissed.
7. Reset	_resetModel() removes node + anchor and resets state flags for re-placement.

4.2 3D Model System

All 3D models are GLB (binary glTF 2.0) files in assets/models/. The ar_flutter_plugin resolves the URI against the native bundle — ios/Runner/ on iOS and android/app/src/main/assets/ on Android. pubspec.yaml declares assets/models/ as a wildcard, so new GLBs are auto-bundled.

File	Size	Monument	Flutter Scale
sphinx.glb	9,436 bytes	Great Sphinx of Giza	scale(0.25, 0.25, 0.25)
khafre_pyramid.glb	1,472 bytes [NEW]	Pyramid of Khafre	scale(0.40, 0.40, 0.40)

The khafre_pyramid.glb geometry: 5 unique positions (4 base corners + apex), 6 triangular faces (4 sides + 2 base), 18 vertices interleaved for flat shading. Base color: sandy limestone [R:0.82, G:0.71, B:0.50, A:1.0]. Metallic 0.0, roughness 0.85. Slope 53.13° matching Khafre's actual pyramid. Height-to-base ratio 0.666.

4.3 Fun Facts Carousel

Each AR screen presents an 8-card horizontal PageView carousel. Every card (_Fact) has: an emoji badge, a bold headline, a ~60-word detail, a 'wow label' badge, and an accent Color. PageController uses viewportFraction=0.90 for peeking. A Set _readFacts tracks swiped cards; dot indicators animate from white

to the card's accent color when read, motivating discovery of all 8.

5. Monument Database

5.1 MonumentInfo Data Model

Field	Description
id	Unique key (e.g. 'khafre_pyramid'). Used for asset paths, routing, Firestore records.
name	Display name (e.g. 'Pyramid of Khafre').
era	Historical period (e.g. 'Old Kingdom, c. 2530 BCE').
location	Geographic location for display.
overview	~80-word summary on monument info cards.
history	~120-word historical narrative; fallback audio narration.
funFacts	List — facts for non-AR contexts.
emoji	Single emoji for lists and badges.
aliases	List — all Vision API name variants. Critical for matching.
customAudioNarrationText	Optional ~250-word TTS override. Falls back to overview.
customTimeline	Optional List for the timeline widget.
assetImage (getter)	Derived: 'assets/monuments/{id}.jpg'.

5.2 Registered Monuments (19 total)

#	ID	Display Name	Status
1	sphinx	The Great Sphinx	Dedicated AR screen + GLB
2	khafre_pyramid [NEW]	Pyramid of Khafre	Dedicated AR screen + GLB
3	pyramids_giza	Pyramids of Giza	Dataset entry
4	great_temple_ramesses	Abu Simbel	Dataset entry
5	temple_isis_philae	Temple of Isis at Philae	Dataset entry
6	karnak_temple	Karnak Temple	Dataset entry
7	luxor_temple	Luxor Temple	Dataset entry
8	colossi_memnon	Colossi of Memnon	Dataset entry
9	pyramid_djoser	Step Pyramid of Djoser	Dataset entry
10	egyptian_museum	Egyptian Museum Cairo	Dataset entry

11	citadel_saladin	Citadel of Saladin	Dataset entry
12	tutankhamun_mask	Mask of Tutankhamun	Dataset entry
13	nefertiti	Nefertiti Bust	Dataset entry
14	hatshepsut	Temple of Hatshepsut	Dataset entry
15	temple_kom_ombo	Temple of Kom Ombo	Dataset entry
16	ramesseum	The Ramesseum	Dataset entry
17	bent_pyramid	Bent Pyramid of Sneferu	Dataset entry
18	colossal_ramesses_ii	Colossal Statue of Ramesses II	Dataset entry
19	akhenaten	Akhenaten / Amenhotep IV	Dataset entry

5.3 NEW Monument — Pyramid of Khafre (khafre_pyramid)

Era: Old Kingdom, c. 2530 BCE **Location:** Giza Plateau, Cairo, Egypt

The Pyramid of Khafre is the second largest pyramid at Giza, built by Pharaoh Khafre (son of Khufu, Fourth Dynasty). It is uniquely identifiable by the intact Tura limestone casing that still crowns its apex. Its steeper 53.13° slope (matching the 3-4-5 Pythagorean triangle) creates the illusion of greater height than the Great Pyramid beside it. The Great Sphinx is widely believed to bear Khafre's face and stands guard along his causeway.

Aliases Registered for Vision API Matching

- Pyramid of Khafre
- Khafre Pyramid
- Pyramid of Chephren
- Chephren Pyramid
- Second Pyramid of Giza
- Khafre
- Chephren

8 Curated AR Fun Facts

#	Headline	Wow Label
1	Second largest pyramid — retains white limestone cap at apex	TOWERING GIANT
2	Still wears original Tura limestone casing near the top	ANCIENT SURVIVOR
3	Aligned within 0.056° of true north via naked-eye astronomy	COSMIC PRECISION
4	The Great Sphinx is believed to bear Khafre's face	ROYAL GUARDIAN
5	Built by paid skilled workers — not enslaved people	MYTH BUSTED
6	Internal temperature stays constant at 20°C year-round	NATURAL AC
7	Slope is exactly 53.13° — a 3-4-5 Pythagorean triangle	MATHEMATICAL
8	Burial chamber was already plundered before Belzoni's 1818 entry	ANCIENT HEIST

Custom Timeline (6 Events)

c. 2558 BCE	Khafre becomes Pharaoh after Khufu.
c. 2530 BCE	Construction begins — ~20,000 workers quarry limestone.
c. 2510 BCE	Pyramid completed — sealed with white Tura casing; Sphinx carved.
1226 CE	Arab merchant inscription — tomb already plundered.
1818 CE	Belzoni officially opens the burial chamber, finds empty sarcophagus.
Present	CulturaX unlock — AR places the pyramid in your world.

6. Gamification System

6.1 GameProgressService

GameProgressService (Provider singleton) tracks user achievements. scanMonument(String id) awards points for first scans, checks badge unlocks, and persists to Firestore under the user's UID. Tracks total points, scan count, badges, and collected artifacts.

6.2 Treasure Hunt

TreasureHuntScreen iterates a monument's treasureHuntMissions. Each TreasureHuntMission has: id, monumentId, title, clue, hint, correctHotspotId, rewardPoints, badgeReward, artifactReward. Tapping the correct hotspot awards points and the artifact. Default: 2 missions per monument.

6.3 AR Hotspots

ARHotspot objects define interactive overlay markers with id, title, description, position (Offset), HotspotType (architecture/history/story/symbol), and icon. Default hotspots: Architectural Details, Historical Origin, Cultural Legend, Symbolic Significance.

7. Asset Structure

```
assets/
  ■■■ models/
    ■ ■■■ sphinx.glb          (9,436 bytes – Great Sphinx 3D model)
    ■ ■■■ khafre_pyramid.glb (1,472 bytes – Pyramid of Khafre) [NEW]
    ■ ■■■ model.tflite        (TFLite placeholder, reserved)
    ■ ■■■ labels.txt          (class labels)
  ■■■ monuments/
    ■■■ sphinx.jpg            + sphinx_ref0..7.jpg          (9 images)
    ■■■ khafre_pyramid.jpg    + khafre_pyramid_ref0..7.jpg (9 images) [ACTIVE]
    ■■■ [16 other monuments × 9 images each]

tools/
  ■■■ generate_pyramid_glb.py [NEW – GLB generator, stdlib only]
```

All asset directories are declared in `pubspec.yaml` as wildcards (`assets/`, `assets/models/`, `assets/monuments/`), so files added to them are auto-bundled without `pubspec.yaml` changes.

8. Key Screens & Navigation

```
Heritage Home Screen
  ■■■ ARMonumentScannerScreen (camera scanner + Vision API)
    ■■■ [sphinx detected]      → SphinxARScreen
    ■■■ [khafre_pyramid]      → KhafrePyramidARScreen [NEW]
    ■■■ [any other monument] → SphinxARScreen (fallback)
    ■■■ ARExperienceScreen     (hotspot overlay on captured image)
    ■■■ TreasureHuntScreen     (from any AR screen)
```

■ ARMonumentScannerScreen

Full-screen camera preview with animated scan frame (bracket corners + scan line via `CustomPainter`). Scan button → Vision API → 4-pass matching → success/failure overlay. Success overlay: monument name, confidence %, 'Explore in AR', 'Start Treasure Hunt', 'Scan Again'. `_resolveARScreen()` dispatches to the correct AR widget; caches captured bytes for the hotspot overlay.

■ SphinxARScreen

AR experience for the Great Sphinx. Loads `sphinx.glb` at scale 0.25. 8 fun facts. `AppColors.gold` accent. 'The Sphinx has risen!' banner.

■ KhafrePyramidARScreen [NEW]

AR experience for the Pyramid of Khafre. Loads `khafre_pyramid.glb` at scale 0.40. 8 curated pyramid facts. Pyramid gold (`#DFAF58`) accent. 'The Pyramid of Khafre has risen!' banner. Independent widget mirroring `SphinxARScreen`'s architecture.

■ TreasureHuntScreen

Gamified mission screen launched from any AR experience. Renders hotspot markers, presents mission clues/hints, awards points on correct selection.

9. Technical Implementation Details

9.1 GLB Model Generation (Python, no external libraries)

khafre_pyramid.glb was created by tools/generate_pyramid_glb.py — a self-contained Python 3 script using only stdlib (struct, json, math, os), following the binary glTF 2.0 spec:

- Compute 5 unique vertex positions (4 base corners + apex).
- For each of 6 triangular faces, compute a flat face normal via cross product + normalize.
- Pack 18 float32 vertices and 18 float32 normals into little-endian binary buffers.
- Pack 18 uint16 indices into the index buffer; pad to a 4-byte boundary.
- Build the glTF JSON chunk: asset, scene, node, mesh, PBR limestone material, 3 accessors, 3 bufferViews, 1 buffer.
- Write the 12-byte GLB header + JSON chunk (space-padded) + BIN chunk.
- Output: a 1,472-byte valid binary glTF 2.0 file.

9.2 AR Node Loading

ARNode uses type: `NodeType.localGLTF2` with uri set to only the filename (e.g., 'khafre_pyramid.glb'). The native bridge resolves this against the iOS bundle root and the Android assets root. The model's position (0,0,0) is relative to the `ARPlaneAnchor`'s world transform, so it sits on the detected surface.

9.3 4-Pass Fuzzy Monument Matching

Pass order is strict-to-loose to prevent false positives. Example: API returns 'Pyramid of Chephren' — Pass 1 fails, Pass 2 succeeds (it is a registered alias, contained in query). Example 2: 'Khafre's Pyramid at Giza' — Passes 1-2 fail, Pass 3 succeeds ('khafre' >4 chars found in query). This achieves high recall on real Vision responses while keeping precision high.

9.4 Cross-Platform Camera Unification

Both capture paths (`CameraController.takePicture()` native, `WebCamController.capture()` web) return `UInt8List` bytes passed directly to `_scan()`. No platform-specific code exists in the detection pipeline beyond the initial capture; `klsWeb` is checked once in `_capture()`.

9.5 Animation System

Each AR screen uses four `AnimationControllers`: `_pulseCtrl` (2s repeating — scan ring pulse), `_slideCtrl` (600ms elastic-out — HUD slide-up), `_bannerCtrl` (700ms elastic-out — welcome banner), `_factPopCtrl` (400ms elastic-out — active card pop on page change). All are disposed in `dispose()` to prevent leaks.

10. Dependencies

Package (pubspec.yaml)	Purpose
ar_flutter_plugin: ^0.7.3	Core AR — ARKit/ARCore, plane detection, GLB nodes
vector_math: ^2.1.4	3D math for ARNode scale (Vector3) and rotation (Vector4)
camera: ^0.11.1	Native camera controller — high-res JPEG capture
image_picker: ^1.2.1	Gallery picker for scanning from photos
flutter_tts: ^4.2.0	Text-to-speech for monument audio narration
confetti: ^0.8.0	Particle confetti on AR placement
firebase_core: ^4.5.0	Firebase SDK initializer
firebase_auth: ^6.2.0	User authentication
cloud_firestore: ^6.1.3	NoSQL database for user progress
provider: ^6.1.5+1	DI + state management
google_fonts: ^8.0.2	Playfair Display + Inter fonts
smooth_page_indicator: ^1.2.0+3	Carousel page dots
mobile_scanner: ^7.2.0	QR/barcode scanning
http: ^1.2.0	HTTP client for Vision API
shared_preferences: ^2.3.0	Local key-value storage
intl: ^0.20.2	i18n and date formatting

11. Future Extension Guide

The architecture is designed for easy addition of new monuments with full AR experiences. Follow these five steps to add one at the same level as the Sphinx and Pyramid of Khafre:

1

Step 1 — Dataset Entry

In `lib/services/monument_data_service.dart` add a new `MonumentInfo(...)` to the monuments list. Required: `id`, `name`, `era`, `location`, `overview`, `history`, `funFacts`, `emoji`, `aliases`. Recommended: `customAudioNarrationText`, `customTimeline`.

2

Step 2 — Reference Images

Add 9 images to `assets/monuments/`: `{id}.jpg` + `{id}_ref0..7.jpg`. No `pubspec.yaml` change needed.

3

Step 3 — AR Screen

Copy `lib/screens/tourist/khafre_pyramid_ar_screen.dart` to `{id}_ar_screen.dart`. Replace the `_kFacts` list, `accent color`, `banner text`, `GLB uri`, `scale Vector3`, and `class name`. No structural changes.

4

Step 4 — 3D GLB Model

Use `tools/generate_pyramid_glb.py` as a template. Modify `POSITIONS/FACES` for your geometry, adjust `base_color`, update `out_path`. Run it; the GLB lands in `assets/models/{id}.glb`.

5

Step 5 — Scanner Routing

In `ar_monument_scanner_screen.dart` import the new screen and add a case to `_resolveARScreen()`: `case '{id}': return YourARScreen(monument: monument);`. No other files change.

CulturaX Technical Report — End of Document

Generated June 2026. Flutter 3.11.1 | `ar_flutter_plugin` 0.7.3 | Google Cloud Vision API | Firebase